

CloudStore

Cloud-Based File Storage System

A secure web application for personal file storage with admin oversight, deployed on an Ubuntu Server virtual machine.

Document	Project Documentation
Version	1.0
Date	May 15, 2026
Audience	Team members, reviewers, future maintainers

1. Project Overview

CloudStore is a Flask-based web application that lets authenticated users upload, download, organise, and delete their own files. It is designed to run on an Ubuntu Server virtual machine (VirtualBox or VMware) so the platform mirrors a realistic cloud deployment — host-to-VM networking, Linux filesystem layout, and a real WSGI server stack in production.

The system enforces a two-tier role model. Regular users only ever see their own files. Administrators have a separate dashboard for managing every user and every file in the system. Authorisation is enforced at the route layer — never by hiding buttons in the UI.

Key capabilities

- Account signup and login with bcrypt-hashed passwords
- Per-user file upload, download, and delete
- File metadata stored in a relational database; raw bytes on the filesystem
- Admin dashboard: list all users, promote/demote, delete users and files
- Category management (admin-created tags applied at upload time)
- CSRF protection on every form (Flask-WTF)
- Production-ready deployment path with Gunicorn + Nginx

2. Technology Stack

Layer	Choice	Notes
Language	Python 3.10+	Tested on 3.12
Web framework	Flask 3	App-factory pattern, blueprints
ORM	SQLAlchemy + Flask-SQLAlchemy	Schema in app/models.py
Migrations	Flask-Migrate (Alembic)	Optional — flask init-db works for quick setup
Database	SQLite (dev) → MySQL (prod)	Switch via DATABASE_URL
Auth session	Flask-Login	Login required, remember-me, role gating
Password hashing	bcrypt	Per-user salt
Forms / CSRF	Flask-WTF + WTForms	Validation + CSRF tokens
Templates	Jinja2	No build step
Frontend	Plain HTML/CSS	Single stylesheet, no JS framework
WSGI (prod)	Gunicorn	4 workers behind Nginx
Config	python-dotenv	Secrets in .env, never in config.py

3. Project Structure

The codebase is organised by feature (blueprint) rather than by file type.

```
cloudstore/
|-- app/
|   |-- __init__.py      # App factory + CLI commands
|   |-- extensions.py    # Shared db, csrf instances
|   |-- models.py        # User, File, Category
|   |-- auth/            # signup, login, logout
|   |   |-- forms.py
|   |   |-- routes.py
|   |-- files/           # upload, download, delete (per-user)
|   |   |-- routes.py
|   |-- admin/           # admin dashboard, user/file mgmt
|   |   |-- routes.py
|   |-- services/storage.py # business logic for files on disk
|   |-- utils/decorators.py # @role_required
|   |-- templates/       # Jinja2 (base + auth + files + admin)
|   |-- static/style.css  # single stylesheet
|-- config.py            # Reads .env, exposes Config class
|-- run.py               # Entry point: python run.py
|-- requirements.txt
|-- .env.example         # Copy to .env and fill in
|-- uploads/             # Local dev upload root (gitignored)
|-- migrations/          # Created by `flask db init`
```

4. Data Model

Three SQLAlchemy models cover everything the app needs.

User

Field	Type	Purpose
id	Integer (PK)	Primary key
username	String(64), unique	Display name, must be unique
email	String(120), unique	Login identifier
password_hash	String(255)	bcrypt hash — never the raw password
role	String(16)	'user' or 'admin'
created_at	DateTime	Account creation timestamp

File

Field	Type	Purpose
id	Integer (PK)	Primary key
original_name	String(255)	Filename as the user uploaded it
stored_name	String(300)	<uuid>_<sanitised> name on disk

relative_path	String(500)	<user_id>/<stored_name> — portable across hosts
size	Integer	Bytes on disk
mime_type	String(100)	Browser-reported content type
uploaded_at	DateTime	Upload timestamp (indexed)
owner_id	FK -> users.id	Required — drives ownership checks
category_id	FK -> categories.id	Optional

Category

Field	Type	Purpose
id	Integer (PK)	Primary key
name	String(64), unique	Admin-managed label, e.g. 'Documents'

5. Roles & Access Rules

Every protected route checks two things: that the request is authenticated, and that the current user is allowed to act on the target resource. The front-end never makes a security decision on its own.

Capability	user	admin
Sign up / log in	Yes	Yes
Upload files	Yes (own bucket)	Yes
Download own files	Yes	Yes
Download any file	No	Yes
Delete own files	Yes	Yes
Delete any file	No	Yes
List all users	No	Yes
Promote / demote users	No	Yes (cannot demote self)
Delete users	No	Yes (cannot delete self)
Manage categories	No	Yes

Enforcement

- **@login_required** on every authenticated route.
- **@role_required('admin')** decorator in `app/utils/decorators.py` for admin-only routes.
- Per-request ownership check: `file.owner_id != current_user.id` and not `current_user.is_admin` → 403.
- Admin blueprint has a `before_request` guard so even adding a new route can't accidentally leak.

6. Route Map

Method	Path	Purpose
GET	/	Redirect to dashboard or login
GET, POST	/auth/signup	Create account
GET, POST	/auth/login	Log in
GET	/auth/logout	Log out (login required)
GET	/dashboard	User's own files + upload form
POST	/upload	Upload a file
GET	/download/<id>	Download a file (ownership check)
POST	/delete/<id>	Delete a file (ownership check)
GET	/admin/	Admin dashboard (counts, links)
GET	/admin/users	List all users
POST	/admin/users/<id>/role	Promote / demote a user
POST	/admin/users/<id>/delete	Delete a user and their files
GET	/admin/files	List every file in the system
GET, POST	/admin/categories	List / create categories

7. How File Uploads Work

File handling is the most security-sensitive part of the application. The flow below is enforced by `app/services/storage.py`:

- Sanitise the filename with `werkzeug.utils.secure_filename` — never trust the browser.
- Generate a random UUID hex and prepend it to the sanitised name → `<uuid>_<original>`. This prevents collisions and makes guessing other users' paths effectively impossible.
- Save under `UPLOAD_FOLDER/<user_id>/<stored_name>`. The per-user directory is created on demand.
- Persist only the *relative* path in the DB — so the same DB row works whether the app runs on Windows dev or in `/var/cloudstore/uploads` on the VM.
- Downloads are served via `send_from_directory` with `as_attachment=True` and the original filename restored for the user.

8. Local Setup

Windows (PowerShell)

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
Copy-Item .env.example .env          # then edit SECRET_KEY
$env:FLASK_APP = "run.py"
flask init-db
flask create-admin admin admin@example.com "ChangeMe123!"
python run.py
```

Linux / Ubuntu

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env          # then edit SECRET_KEY
export FLASK_APP=run.py
flask init-db
flask create-admin admin admin@example.com 'ChangeMe123!'
python run.py
```

Then open `http://127.0.0.1:5000`.

9. VM Deployment

Target: Ubuntu Server 22.04 LTS in VirtualBox or VMware.

- Set the VirtualBox adapter to **Bridged** so the host browser can reach the VM IP directly.
- Create the upload root once: `sudo mkdir -p /var/cloudstore/uploads && sudo chown -R $USER /var/cloudstore`
- Point `UPLOAD_FOLDER` in `.env` at `/var/cloudstore/uploads`.
- Switch `DATABASE_URL` to MySQL: `mysql+pymysql://user:pwd@localhost/cloudstore`
- Run with Gunicorn: `gunicorn -w 4 -b 127.0.0.1:8000 run:app`
- Front with Nginx as a reverse proxy on port 80.
- Firewall: `sudo ufw allow 80 (prod)` or `sudo ufw allow 5000 (dev only)`.

10. Security Notes

- Passwords are bcrypt-hashed with a unique salt per user. Plain text never touches the database.
- All forms include a CSRF token, enforced globally by `CSRFProtect`.
- Filenames are sanitised before saving — directory traversal and dotfile tricks are blocked.
- `SESSION_COOKIE_HTTPONLY` = `True` and `SAMESITE` = `'Lax'` to mitigate XSS / CSRF cookie theft.
- Secrets live in `.env` (gitignored), loaded by `python-dotenv`. **Never commit a real .env.**
- Login redirects validate that `next=` targets are same-origin — no open redirects.

- Admins can't delete or demote themselves — prevents accidental logout.

11. Anti-Patterns to Avoid

- Do NOT store an absolute OS path in the DB — store the relative path from `UPLOAD_FOLDER`.
- Do NOT pass user input straight into `os.path.join(UPLOAD_FOLDER, ...)`. Always sanitise first.
- Do NOT skip the ownership check just because an ID 'isn't exposed in the UI' — IDs are guessable.
- Do NOT put secrets into `config.py`. They belong in `.env`.
- Do NOT rely on hiding admin links in templates as a security measure — gate the routes.
- Do NOT use `print()` for user-facing messages — use `flash()`.

12. Future Work

- **Storage usage stats:** per-user quota with a progress bar on the dashboard.
- **File sharing:** generate a signed share token, add a `shared_files` join table.
- **Preview pane:** inline render images and PDFs without forcing a download.
- **Two-factor auth:** TOTP via `pyotp` for admin accounts.
- **Audit log:** record every admin action (role change, user delete, file delete) for compliance.
- **Chunked uploads:** support files larger than `MAX_CONTENT_LENGTH`.

13. Glossary

Term	Meaning
Blueprint	Flask's modular grouping of routes — we have one per feature.
App factory	<code>create_app()</code> function that builds the Flask app — enables config swap
Role gating	Server-side check that the logged-in user has permission to call a route.
UUID prefix	Random 32-char hex prepended to filenames to prevent collisions and guessing.
WSGI	The Python web server protocol — Gunicorn speaks it to Nginx.

Generated automatically from `docs/generate_pdf.py`. Re-run that script whenever the source of truth changes.